S Sentra

User & Access Management Platform

Django REST API + React admin console — build brief

1 developer / deliver to abdulvahab@prodevinfo.com

The Assignment

01 · BACKEND

Django REST API

JWT authentication, user management, roles & permissions, role-based access control on every endpoint, audit logging.

02 · FRONTEND

React Admin Console

Modern UI/UX: login & create account, dashboard, user CRUD, role assignment, permission matrix, audit log — tables with filters, sort, pagination and Excel export.

✓ Public GitHub repo + README **✓ Deployed on Render (free)** **✓ Tests, security & CI included**

Architecture

RENDER STATIC SITE

React 19 + Vite 8

React Router 7 · TanStack Query 5 · axios

JWT ⇄

RENDER WEB SERVICE

Django 6 + DRF 3.16

SimpleJWT · gunicorn · whitenoise

ORM ⇄

RENDER POSTGRES

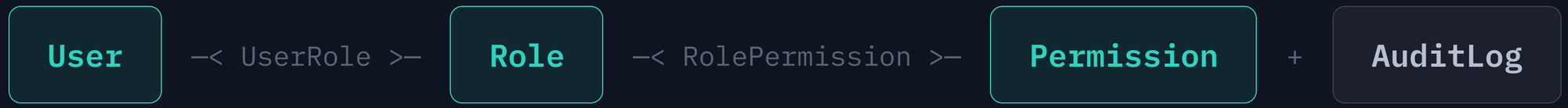
PostgreSQL 18

free tier

- Access token: 15 min, memory only
- Refresh token: httpOnly cookie, rotated + blacklisted
- Monorepo: /backend + /frontend + render.yaml

Data Model & RBAC Vocabulary

User → roles → permissions. First-class app tables — the API owns the vocabulary.



PERMISSION CODENAMES

`users.view` · `users.create` · `users.edit` · `users.delete`

`roles.view` · `roles.manage` · `permissions.view` · `audit.view`

SEED ROLES

Admin — everything, delete-protected

Manager — manage users, view roles

Viewer — read-only

API Surface

All under `/api/v1/` — no endpoint ships without an explicit permission.

`POST /auth/login · refresh · logout`	anon / cookie	`GET /auth/me`	authenticated
`CRUD /users`	**users.***	`PUT /users/{id}/roles`	**roles.manage**
`CRUD /roles`	**roles.view / manage**	`PUT /roles/{id}/permissions`	**roles.manage**
`GET /permissions`	**permissions.view**	`GET /audit-logs`	**audit.view**

List contract on every table: ?search · field filters · ?ordering · ?page — plus GET …/export/ streaming an .xlsx of the current filter set.

Security Requirements

Tokens & sessions

Argon2 hashing. Refresh rotation with blacklist. Logout truly invalidates. No tokens in localStorage.

Hardened surface

Login throttling. Strict CORS allow-list. HSTS + security headers. Generic login errors — no user enumeration.

Guard rails

Can't deactivate yourself or drop your own Admin role. System roles delete-protected. Every mutation audited.

The test that matters: every seed role × every endpoint → assert the expected 200 or 403.

Frontend Screens

The interactive prototype shipped with this brief is the UI spec — build to it.

`/login`

Login / Create account

Inline validation, throttle-aware errors

`/`

Dashboard

Stats + recent activity feed

`/users`

Users

Filters, sort, server-side pagination, Excel export

`/roles`

Roles & Permissions

Role list + clickable permission matrix

`/audit`

Audit Log

Append-only; filters, sort, pagination, export

`<Can perm="…">`

Permission gating

UI hides what you can't do; API still enforces

Quality Gates

Lint

ruff + black (backend), ESLint + Prettier (frontend)

Tests

pytest ≥80% on auth/RBAC; vitest on auth hooks & gating

Dependencies

pip-audit + npm audit in CI, Dependabot on

Performance

pagination, prefetch_related, DB indexes, route code-splitting

GitHub Actions · runs on every push · main must stay green

Work Plan

PHASE 1

Models + Auth

Scaffold, custom User, RBAC tables, seeds, JWT endpoints

PHASE 2

RBAC API

Viewsets, HasPermission, list contract + xlsx export, matrix tests, Swagger

PHASE 3

React UI

Auth + signup, shell, DataTables (filter/sort/page/export), roles matrix

PHASE 4

Ship

Deploy on Render, smoke test, README, email the repo

Buffer rule: if Phase 3 slips, cut dashboard stat cards — never tests, security config, or deployment.

Definition of Done

01 Public GitHub repo — README with 5-command setup, demo credentials, live URLs

02 Live on Render: frontend, API, and Swagger UI all reachable

03 Admin, Manager, Viewer behave differently in the UI — and get 403s at the API

04 CI green on main: lint, tests, coverage gate

05 Emailed to abdulvahab@prodevinfo.com by **July 6, 2026**